

מדינת ישראל

משרד החינוך

מינהל מדע וטכנולוגיה

הנדסת תוכנה וסייבר

ספר פרויקט גמר

SecureSync

מערכת מאובטחת לאחסון והעברת קבצים בסביבות ארגוניות

שם התלמיד: שם פרטי ומשפחה]

מספר ת.ז.: מספר]

שם המורה המנחה: שם]

שם המוסד: שם בית הספר /המכללה]

תאריך הגשה: מאי 2026

תוכן עניינים

1. הצעת הפרויקט המאושרת על ידי משרד החינוך
2. תקציר /מבוא
 - 2.1 רקע לפרויקט
 - 2.2 תהליך המחקר
 - 2.3 סקירת ספרות
 - 2.4 אתגרים מרכזיים
3. מטרות /יעדים
4. אתגרים
5. מדדי הצלחה למערכת
6. רקע תיאורטי /ספרות מקצועית
7. תיאור פתרונות קיימים לבעיה
8. ניתוח חלופות מערכתי
9. תיאור החלופה הנבחרת עם נימוקים
10. אפיון המערכת המוצעת /המוגדרת
11. תיאור הארכיטקטורה
12. תיאור תהליכי אבטחת מידע במערכת
13. (Anomaly Detection) למידת מכונה
14. תיאור התוכנה
15. UML / Use Cases ניתוח ותרשימי
16. (Screen Flow Diagram) תרשים מסכים
17. פירוט מסכים
18. אלמנטי תצוגה
19. הודעות למשתמש
20. ממשק משתמש
21. קוד התוכנית עם תיעוד
22. תיאור מסד הנתונים
23. מדריך למשתמש
24. בדיקות והערכה
25. מסקנות
26. פיתוחים עתידיים
27. ביבליוגרפיה

הצעת הפרויקט המאושרת על ידי משרד החינוך 1.

מערכת מאובטחת לאחסון והעברת קבצים בסביבות ארגוניות — SecureSync: שם הפרויקט

תחום: הנדסת תוכנה, אבטחת מידע וסייבר

Zero-Knowledge עם ארכיטקטורת (E2E) המממש הצפנה מקצה לקצה full-stack תיאור קצר: פרויקט כולל זיהוי חריגות בלמידת מכונה, ניהול הרשאות, גרסאות קבצים, ואודיט מלא.

סטטוס: הצעה מאושרת

2. תקציר / מבוא

2.1 רקע לפרויקט

נולד מהצורך של ארגונים להעביר קבצים רגישים — מסמכים משפטיים, נתוני לקוחות, SecureSync, קניין רוחני — מבלי לסמוך על ספקי ענן שעשויים לסרוק את התוכן. הבעיה המרכזית: כאשר ארגון מעלה קבצים לשרת ענן, הוא מאבד שליטה על מי שיכול לקרוא אותם. גם אם ישנה הצפנה "בזמן מנוחה", ספק הענן מחזיק את מפתחות ההצפנה ויכול לפענן כל קובץ. הפתרון: הצפנה בצד הלקוח בלבד — השרת לא מקבל לעולם קוד מפתח או תוכן גלוי.

2.2 תהליך המחקר

המחקר המקדים בחן את הגישות הקיימות לאחסון קבצים מאובטח:

- המובנה בדפדפן WebCrypto API לעומת crypto-js כגון JavaScript ספריות
 - AES-256-GCM (AEAD) לעומת AES-128-CBC: אלגוריתמי הצפנה
 - Key Management: RSA-OAEP (Key Wrapping) עם AES הצפנת מפתח
 - ML (Isolation Forest, SVM) לעומת (Z-score, EWMA) שיטות זיהוי חריגות: סטטיסטיקות
- WebCrypto + RSA נבחרה גישה, לאחר בחינת הפתרונות הקיימים (ProtonDrive, Tresorit, Keybase), אמיני Zero-Knowledge כמאפשרת Key Wrapping.

2.3 סקירת ספרות

מקורות עיקריים שנסקרו:

- MDN Web Docs — Web Crypto API documentation
- NIST SP 800-38D — Recommendation for GCM mode
- RFC 7519 — JSON Web Tokens (JWT)
- OWASP Top 10 — Web Application Security Risks
- "Machine Learning for Anomaly Detection" — Chandola et al., ACM Computing Surveys
- MongoDB Mongoose documentation
- scikit-learn documentation (Isolation Forest, One-Class SVM)

2.4 אתגרים מרכזיים

2.4.1 הבעיה שעמה התמודד התלמיד

הבעיה המרכזית: כיצד לבנות מערכת אחסון קבצים שגם אם השרת נפרץ, התוקף לא יוכל לקרוא שום מלא — השרת Zero-Knowledge קובץ, ולא יידע שם קובץ, סוג, גודל, או תוכן. הדרישה הטכנית היא plaintext ואף פעם לא ciphertext מאחסן רק.

2.4.2 סיבות לבחירת הנושא

הנושא נבחר בשל:

- עניין אישי עמוק בתחום אבטחת המידע והסייבר
- (חוק הגנת הפרטיות בישראל, GDPR) הרלוונטיות הגוברת של פרטיות מידע

- ML + full-stack + האתגר הטכני המשמעותי: שילוב קריפטוגרפיה
- Zero-Knowledge + Anomaly Detection הצורך הארגוני האמיתי — פתרונות קיימים לא מציעים

2.4.3 מוטיבציה לעבודה

מקצועי, שמשלב ידע מתחומים מרובים: portfolio-המוטיבציה נבעה מרצון לממש פרויקט שניתן להציג כ מסדי נתונים, ולמידת מכונה. הפרויקט מדמה אתגרים אמיתיים, frontend, backend, קריפטוגרפיה ואנליטיקת, AWS IAM-ניהול גישה כמו ב, WhatsApp/Signal-כמו ב E2E מהתעשייה — הצפנה SIEM-אנומליות כמו ב

2.4.4 על איזה צורך עונה הפרויקט? איזה פתרון הוא נותן?

הצורך: ארגונים קטנים ובינוניים שצריכים שיתוף קבצים מאובטח ללא תלות בספקי ענן גדולים בדפדפן, כך שרק המשתמש המורשה יכול AES-256-GCM עם הצפנה on-premise הפתרון: מערכת לפענח את הקובץ — אפילו מנהל השרת לא יוכל לקרוא את תוכנו

2.4.5 הצגת פתרונות שנבחנו במחקר המקדים

- נדחה: השרת מחזיק מפתחות — (Server-side encryption) פתרון 1: הצפנה בשרת
- בקליינט — נדחה: ספרייה חיצונית, ביצועים נמוכים crypto-js: פתרון 2
- אמיתי Zero-Knowledge, מהיר, native: נבחר — WebCrypto API: פתרון 3
- web-לא מתאים ל, UX-נדחה: מורכב ל — PGP encryption: פתרון 4

מטרות / יעדים 3.

מטרה	פרטים	סטטוס
הצפנת קצה-לקצה	בדפדפן לפני העלאה לשרת AES-256-GCM	הושג ✓
Zero-Knowledge	מוצפן metadata — השרת לא רואה שם /סוג /תוכן קובץ	הושג ✓
אימות חזק	שנשלח לאימייל JWT + 2FA OTP	הושג ✓
הרשאות גישה	שיתוף קבצים עם תפקידים read / write / admin	הושג ✓
גרסאות קבצים	שמירת היסטוריית גרסאות לכל קובץ	הושג ✓
אודיט מלא	זמן, תוצאה, IP, רישום כל פעולה עם	הושג ✓
זיהוי חריגות	בזמן אמת WebSocket התראות + ML אלגוריתמי 4	הושג ✓
ניהול מכשירים	ניטור מכשירים מחוברים, חסימת מכשירים/משתמשים	הושג ✓
TLS 1.3	הצפנת כל התקשורת בין קליינט לשרת	הושג ✓

4. אתגרים

פתרון	אתגר	#
metadata → בנפרד AES(-שם, סוג, גודל (מוצפן ב encryptedMetaB64	Zero-knowledge: שמירת שם קובץ בלי שהשרת ידע	1
עקבי לאימות hash → הסופי ciphertext-על ה hash חישוב	SHA-256 על ciphertext שמשתנה עם כל הצפנה	2
MongoDB-לפני שמירה ל bool() ב boolean עטיפת כל	pymongo לא תואם np.True_	3
מהחלטה הסופית — 3 אלגוריתמים אחרים EWMA הסרת הספיקו	EWMA מייצר False Positives גבוהים	4
npm selfsigned package-שימוש ב	TLS cert generation — openssl לא מותקן	5
token query param → אימות JWT בחיבור	WebSocket אימות	6
returnDocument: "after"-שינוי ל	Mongoose "new: true" deprecated	7
בדף השגיאה "thisisunsafe" הקלדת	self-signed cert דפדפן דוחה	8

מדדי הצלחה למערכת 5.

תוצאה	ערך יעד	מדד
✓ ciphertext בלבד	Zero plaintext על הדיסק	הצפנה: שרת לא רואה תוכן קובץ
✓ encryptedMetaB64	שם/סוג/גודל מוצפן	Zero-Knowledge metadata
✓	מהתחברויות 100%	2FA: OTP לאימייל
✓ Z-score = 3.44 בבדיקות	Z-score > 2.0 → התראה	Anomaly Detection
✓ signature על 403 שגויה	זיהוי שיבוש קובץ	RSA Signature
✓ jti נמחק מ DB	Logout מבטל JWT	Session invalidation
✓ נקודות רישום 7	כל פעולה נרשמת	Audit Log
✓ <500ms latency	התראה בזמן אמת	WebSocket alert

6. רקע תיאורטי / ספרות מקצועית

6.1 מושגים מרכזיים בתחום הדעת

- הוא AES-256-GCM. שימוש במפתח זהה להצפנה ופענוח: (Symmetric Encryption) הצפנה סימטרית NIST. תקן מוביל מאושר.
- RSA-OAEP. מפתח ציבורי להצפנה, פרטי לפענוח: (Asymmetric Encryption) הצפנה אסימטרית Key Wrapping-משמש ל.
- AEAD (Authenticated Encryption with Associated Data): GCM mode authentication tag מוסיף שמונע שיבוש.
- Zero-Knowledge Architecture: ארכיטקטורה שבה הספק לעולם לא מקבל גישה לנתונים בטקסט גלוי.
- JWT (JSON Web Token): stateless. לאימות RFC 7519 תקן HMAC-SHA256. חתומים בסוד claims מכיל.
- Defense in Depth: עקרון אבטחה של שכבות הגנה מרובות — כישלון שכבה אחת לא חושף את המערכת.

6.2 נושאים רלוונטיים בתחום הדעת

- Key Wrapping: (KEK — Key Encryption Key) טכניקה להצפנת מפתח הצפנה אחד עם מפתח אחר.
- IV (Initialization Vector): ciphertext תניב plaintext ערך אקראי המבטיח שהצפנת אותו פעם.
- PBKDF2 / bcrypt: גבוה iteration count-אקראי ו salt מכוונות לאחסון סיסמאות עם hash פונקציות.
- WebSocket: משמש להתראות בזמן אמת, TCP פרוטוקול תקשורת דו-כיוונית מתמשכת מעל.
- Anomaly Detection: (Isolation Forest) ולמידת מכונה (Z-score) זיהוי סטיות מדפוס נורמלי — סטטיסטיקה (Isolation Forest).

6.3 סקירה מקצועית של הבעיה בעולם

רשומות. (הפתרון הנפוץ הוא הצפנה בצד הלקוח: Dropbox (2012, 68M), Capital One (2019, 100M) פרצות נתונים בספקי ענן הן בעיה גוברת

- ProtonDrive, Tresorit Zero-Knowledge מציעות אף ללא Anomaly Detection
- AWS S3 SSE-C מאפשר הצפנה בצד הלקוח, אף הלקוח אחראי לניהול המפתחות
- Signal Protocol — E2E encryption למסרים, לא לקבצים ארגוניים
- SecureSync דשבורד ניהולי + Anomaly Detection + Zero-Knowledge + E2E: ממלא פער

6.4 נושאים רלוונטיים במדעי המחשב הקשורים לבעיה

- גמיש לניהול 20 מודלים שונים schema מאפשר MongoDB document store: מבני נתונים.
- GHASH עם CTR mode שילוב — GCM (Galois/Counter Mode): תיאוריית הצפנה.
- Isolation Forest — למידת מכונה: outliers לזיהוי Tree-based אלגוריתם; One-Class SVM — hyperplane להפרדת נורמלי מאנומלי.

- I/O קריטי לביצועי שרת — Node.js Event Loop, Promise chains, async/await: תכנות אסינכרוני
- HTTP-ב Node.js מתקשר עם ML, נפרד ל Python Flask service: ארכיטקטורת מיקרוסרביס

7. תיאור פתרונות קיימים לבעיה

ניהול ארגוני	Anomaly Detection	Zero-Knowledge	E2E הצפנת	מערכת
חלקי	X	X	X	Google Drive
✓	X	X	X	Dropbox
X	X	✓	✓	ProtonDrive
✓	X	✓	✓	Tresorit
✓	X	X	X	OneDrive
✓	✓	✓	✓	SecureSync

דשבורד ניהולי + Zero-Knowledge + Anomaly Detection ייחודי בשילוב SecureSync: מסקנה ארגוני.

8. ניתוח חלופות מערכת

חלופה	יתרונות	חסרונות	הערכה
Server-side encryption	פשוט לפיתוח	שרת מחזיק מפתחות — לא Zero-Knowledge	נדחתה
PGP / GPG	תקן מוכר	קשה, לא מתאים לדפדפן UX	נדחתה
בקליינט crypto-js	גמיש	ספרייה חיצונית, ביצועים נמוכים	נדחתה
WebCrypto + RSA Key Wrapping	Native, מהיר, Zero-Knowledge אמיתי	מורכבות מימוש גבוהה	✓ נבחרה

9. תיאור החלופה הנבחרת עם נימוקים

החלופה הנבחרת: WebCrypto API + RSA-OAEP Key Wrapping + AES-256-GCM.

נימוקים:

- Native API — מובנה בכל דפדפן מודרני, ללא תלות בספריות חיצוניות
- JavaScript מהיר פי 10 מספריות — browser engine ב ++C-מנוהל ב API-ביצועים: ה
- Zero-Knowledge: המפתח הפרטי לא עוזב את הדפדפן לעולם
- Key Wrapping: עם המפתח AES ייחודי — הצפנת מפתח RSA-2048 כל משתמש מחזיק זוג מפתחות הציבורי
- AEAD: GCM mode authentication tag מוסף שיבוש — ciphertext

10. אפיון המערכת המוצעת /המוגדרת

10.1 ניתוח דרישות המערכת

דרישות פונקציונליות:

#	דרישה	עדיפות
F1	FA רישום משתמש עם אימות אימייל ו-2	גבוהה
F2	JWT session management התחברות עם	גבוהה
F3	(AES-256-GCM) העלאת קובץ עם הצפנה בדפדפן	גבוהה
F4	הורדת קובץ ופענוח אוטומטי בדפדפן	גבוהה
F5	(read/write/admin) שיתוף קובץ עם הרשאות	גבוהה
F6	גרסאות קבצים — העלאת גרסה חדשה	בינונית
F7	מחיקת קובץ	גבוהה
F8	audit logs דשבורד ניהולי עם	בינונית
F9	(WebSocket) התראות בזמן אמת על חריגות	בינונית
F10	ניהול וחסיתת משתמשים/מכשירים	בינונית

דרישות לא-פונקציונליות:

#	דרישה	פרטים
NF1	אבטחה	AES-256-GCM + TLS 1.3 + SHA-256 + RSA
NF2	ביצועים	MB העלאת/הורדה עד 100
NF3	זמינות	לאתחול אוטומטי nodemon עם Node.js שרת
NF4	תחזוקה	ESM modules, Mongoose schemas, קוד מודולרי
NF5	נגישות	UI ללא תלות בספריות HTML/CSS ממשק

10.2 מודל המערכת

המערכת מורכבת מ-3 רכיבים עיקריים:

- תקשורת עם השרת, (WebCrypto) ממשק המשתמש, לוגיקת הצפנה — React קליינט
- WebSocket, ניהול קבצים, אימות, הרשאות, אודיט, REST API — Node.js/Express שרת
- ML ניתוח אנומליות עם 4 אלגוריתמי — Python (Flask) מיקרוסרביס

10.3 אפיון פונקציונלי

המערכת תומכת ב-3 סוגי פעולות עיקריות:

- חסימה, FA, logout, ניהול משתמשים: רישום, התחברות עם 2
- ניהול קבצים: העלאת מוצפנת, הורדה ופענוח, שיתוף, גרסאות, מחיקה
- ניהול ואבטחה: אודיט לוג, זיהוי חריגות, ניהול מכשירים, דשבורד

10.4 ביצועים עיקריים

- (WebCrypto native) ~200ms — MB זמן הצפנה: קובץ 10
- בשרת SHA-256 לאימות ms זמן העלאת: תלוי ברשת +50
- Python service ב-ms ניתוח אנומליה: ~100
- מרגע אנומליה לקבלת התראה בדפדפן <500ms WebSocket latency

אילוצים 10.5

- לענן deployment ריצה מקומית — לא
- Self-signed certificate — דפדפן מציג אזהרה
- Multer 500 ל-MB מוגדר ל-: גודל קובץ מקסימלי
- Python service על port 5001 חייב לרוץ

11. תיאור הארכיטקטורה

11.1 Top-Down Level Design — ארכיטקטורת הפתרון המוצע

SecureSync – Top-Down Architecture

- Authentication Module
 - POST /users/register → bcrypt hash → send OTP (Nodemailer)
 - POST /users/login → verify password → send OTP
 - POST /users/verify-otp → validate → create JWT + Session (jti)
 - POST /users/logout → invalidate Session
- File Management Module
 - POST /files/upload → multer → SHA256 verify → RSA sign → MongoDB
 - GET /files/ → own files + shared files
 - GET /files/:id/download → check access → verify signature → stream
 - DELETE /files/:id → unlink + delete doc + delete permissions
- Permission Module
 - POST /files/:id/share → find user → create Permission
 - DELETE /files/:id/share/:permId → revoke
- Versioning Module
 - POST /files/:id/versions → save new FileVersion
 - GET /files/:id/versions → list
- Security / PKI Module
 - generateKeyPair() → RSA-2048 → KeyPair doc
 - signBuffer() / verifySignature()
- Anomaly Detection Module
 - Node.js: analyzeEvent() → POST http://localhost:5001/analyze
 - Python Flask: Z-score + EWMA + Isolation Forest + One-class SVM
- Monitoring Module
 - AuditLog → every action logged
 - WebSocket → broadcast anomaly alerts
 - Device heartbeat
- Admin Module
 - GET /admin/audit-logs, /anomaly-scores, /sessions, /users
 - POST/DELETE /admin/block

11.2 תיאור הרכיבים בפתרון

פורט	תיאור	סוג	רכיב
5173	WebCrypto ממשק משתמש + הצפנה	Client App	Browser (React)
4000	REST API, JWT, File serving, WebSocket	Application Server	Node.js / Express
Cloud	20 Mongoose schemas, NoSQL document store	Database	MongoDB Atlas
5001	Anomaly detection: Z-score, IF, SVM	ML Microservice	Python Flask

11.3 תיאור תהליכי מערכת ההפעלה

- בפורט 5173 React קליינט — npm run dev (Vite) תהליך 1
- בפורט 4000 Node.js שרת — nodemon index.js תהליך 2 (HTTPS)
- בפורט 5001 Flask ML service — python main.py תהליך 3
- בפורט 5001 Flask ML service — python main.py תהליך 3
- MongoDB Atlas שירות ענן — mongoose.connect() חיבור דרך שירות ענן תהליך 4

11.4 ארכיטקטורת רשת

```
[Browser / Client]
| HTTPS (TLS 1.3) - port 4000
| WSS (WebSocket Secure)
[Node.js Server]
| mongoose.connect (TLS) | HTTP - port 5001
[MongoDB Atlas Cloud] [Python Flask Service]
```

11.5 בארכיטקטורה API תיאור

כל router modules: users, files, permissions, fileVersions, admin, devices. ל-6 API-ה תגובות בפורמט (JWT + jti check + BlockRule check) authRequired middleware מוגן ב route JSON עם HTTP status codes (500, 404, 403, 401, 400, 201, 200). סטנדרטיים

11.6 תיאור פרוטוקולי תקשורת

- HTTPS / TLS 1.3: מוצפנת REST כל תקשורת. self-signed certificate.
- WSS (WebSocket Secure): לאימות token query param. התראות בזמן אמת.
- HTTP (internal): Node.js → Python /analyze endpoint (localhost לחוץ)

11.7 קליינט-שרת

Vite proxy קלאסי עם Client-Server: ארכיטקטורה

- Vite dev server proxy → Node.js: 4000 דרך */api/-הקליינט שולח בקשות ל
- WebSocket ל wss://localhost: 4000-מחובר ישירות ל
- MongoDB: ל Atlas-השרת בלבד מתחבר ל (הנתונים)

12. תיאור תהליכי אבטחת מידע במערכת

12.1 תיאור ההגנה

שכבות הגנה עצמאיות 7 — Defense in Depth מיישמת SecureSync

שכבה	מנגנון	מגן מפני
1	TLS 1.3	ציטוט (eavesdropping), Man-in-the-Middle
2	JWT + 2FA	גניבה לא מורשית, Session hijacking
3	AES-256-GCM	קריאת תוכן קובץ — גם אם השרת נפרץ
4	SHA-256 + RSA Signature	שיבוש קובץ (Tampering), File integrity
5	Permission Model	גישה לא מורשית לקבצים של משתמשים אחרים
6	Anomaly Detection	מתקפות נפח (brute force, data exfiltration)
7	Audit Log	מעקב אחר פעולות חשודות, Forensics

12.2 תיאור ההצפנות

Zero-Knowledge Flow: — זרימת הצפנת קובץ

```
[Client - Browser]
1. generateKeyPair(RSA-2048)           ← פעם אחת למשתמש
2. generateKey(AES-256-GCM)           ← לכל קובץ
3. encrypt(file, AES_key, iv)         → ciphertext
4. encryptMeta({name,type,size})      → encryptedMetaB64
5. wrapKey(AES_key, RSA_publicKey)    → wrappedKeyB64
6. SHA256(ciphertext)                 → hash

[Upload → Server]
→ ciphertext + ivB64 + wrappedKeyB64 + encryptedMetaB64 + ciphertextSha256B64

[Server]
7. verify SHA256(ciphertext) === received hash
8. sign(ciphertext, RSA_privateKey)   → signature
9. save File doc to MongoDB (NO plaintext ever stored)

[Download ← Server]
10. verify signature → detect tampering
11. send ciphertext + x-securesync-meta header

[Client - Decrypt]
12. unwrapKey(wrappedKeyB64, RSA_privateKey) → AES_key
13. decrypt(ciphertext, AES_key, iv)      → file
14. decryptMeta(encryptedMetaB64)        → {name, type}
15. trigger browser download with original filename
```

13. Anomaly Detection — למידת מכונה 13.

13.1 תהליך איסוף נתונים

לכל רשומה. AuditLog collection-מתועדת ב (upload, download, delete, share) כל פעולת משתמש: userId, action, timestamp, IP. ל 59: לוגים היסטוריים הוכנסו ידנית ל baseline-MongoDB בבדיקות: לפני הרצת הניתוח.

13.2 איזון ואופטימיזציה של נתונים

- עם פעולות נורמליות AuditLog רשומות 59: baseline נתוני
- userId+action לכל std deviation-מחשב ממוצע ו Python service-ה
- גבוהים — 3 אלגוריתמים נותרו False Positives הוסר מהחלטה הסופית כי יצר EWMA

13.3 סוג הלמידה שנבחר

למידה לא מפקחת. אין תיוג ידני של "אנומליה" / "נורמלי". האלגוריתמים — Unsupervised Learning לומדים את הדפוס הנורמלי ומזהים סטיות.

13.4 רשתות למידה ונוסחאות

אלגוריתם	נוסחה / עקרון	ספרייה
Z-score	$z = (x - \mu) / \sigma$; anomaly if $z > 2.0$	numpy
EWMA	(לצורך מעקב, לא החלטה) $S_t = \alpha \cdot x_t + (1-\alpha) \cdot S_{t-1}$	numpy
Isolation Forest	קצר outlier = isolation depth — עצי הפרדה אקראיים	scikit-learn
One-Class SVM	hyperplane RBF kernel המפריד normal מ-anomaly	scikit-learn

anomaly = (z_score > 2.0) OR isolation_forest_anomaly OR svm_anomaly: החלטה סופית

13.5 מוניטורינג ואופטימיזציה של מדדי ביצועים

- AnomalyScore collection: zScore, ewma, anomaly, details-כל תוצאת ניתוח נשמרת ב
- "Anomaly Scores" tab-הדשבורד מציג את כל הניקודים ב
- anomaly=true של התראה לכל המשתמשים המחוברים כאשר broadcast מבצע WebSocket

14. תיאור התוכנה

14.1 API פירוט

Response	תיאור	Auth	Route	Method
{message: "OTP sent"}	רישום משתמש	✗	/users/register	POST
{message: "OTP sent"}	התחברות + שליחת OTP	✗	/users/login	POST
{token: "JWT..."}	OTP אימות	✗	/users/verify-otp	POST
{message: "Logged out"}	session ביטול	✓	/users/logout	POST
{id: "fileId"}	העלאת קובץ מוצפן	✓	/files/upload	POST
[File...]	רשימת קבצים	✓	/files/	GET
binary + x-securesync-meta	הורדת קובץ	✓	/files/:id/download	GET
{message: "Deleted"}	מחיקת קובץ	✓	/files/:id	DELETE
{permissionId}	שיתוף קובץ	✓	/files/:id/share	POST
{message}	ביטול שיתוף	✓	/files/:id/share/:id	DELETE
{versionId}	גרסה חדשה	✓	/files/:id/versions	POST
[FileVersion...]	רשימת גרסאות	✓	/files/:id/versions	GET
[AuditLog...]	כל הפעולות	✓	/admin/audit-logs	GET
[AnomalyScore...]	ניקודי חריגה	✓	/admin/anomaly-scores	GET
{ruleId}	חסימת מכשיר/משתמש	✓	/admin/block	POST
{anomaly, zScore, details}	ניתוח אנומליה	✗	/analyze (Python: 5001)	POST

14.2 סביבת פיתוח

גרסה	כלי / טכנולוגיה	רכיב
—	Windows 11	מערכת הפעלה
—	Visual Studio Code	עורך קוד
18.x	JavaScript (React 18)	שפת קליינט
22 / 5.2.1	Node.js + Express	שפת שרת
3.11	Python	שפת ML
—	MongoDB Atlas	מסד נתונים
latest	Vite	Build Tool
—	Git	ניהול גרסאות
—	Postman / Browser DevTools	בדיקות API

14.3 שפות תכנות

- JavaScript (ES2022+): React + שרת Node.js/Express. ESM modules.
- Python 3.11: Flask microservice. לאנומלי דיטקשן numpy, scikit-learn, pymongo.
- CSS: חיצוני framework עיצוב ממשק המשתמש. ללא.

15. UML / Use Cases ניתוח ותרשימי

15.1 UML תיאור האלגוריתם הראשי + תרשימי

האלגוריתם המרכזי: הצפנת קובץ + העלאה מאובטחת

```
algorithm encryptAndUpload(file):  
  1. aesKey ← crypto.subtle.generateKey(AES-GCM, 256)  
  2. iv ← crypto.getRandomValues(12 bytes)  
  3. ciphertext ← crypto.subtle.encrypt({AES-GCM, iv}, aesKey, file)  
  4. metaJson ← JSON({name, type, size})  
  5. metaIv ← crypto.getRandomValues(12 bytes)  
  6. encMeta ← crypto.subtle.encrypt({AES-GCM, metaIv}, aesKey, metaJson)  
  7. wrappedKey ← crypto.subtle.wrapKey(aesKey, rsaPublicKey, RSA-OAEP)  
  8. hash ← SHA256(ciphertext)  
  9. formData ← {ciphertext, iv, wrappedKey, encMeta, metaIv, hash, algorithm}  
  10. POST /files/upload(formData)  
  return fileId
```

15.2 Use Case הצגת לכל הפונקציות העיקריות

UC	שם	Actor	Pre-condition	תמצית (Main Flow)
UC-01	FA התחברות 2	משתמש	רשום במערכת	email+pass → OTP → JWT
UC-02	העלאת קובץ מוצפן	משתמש מחובר	תקף JWT	AES encrypt → SHA256 → POST → sign → save
UC-03	הורדת קובץ	בעלים/בעל הרשאה	תקף, קובץ JWT קיים	verify sig → stream → RSA unwrap → AES decrypt
UC-04	שיתוף קובץ	בעלים	תקף JWT	email lookup → create Permission doc
UC-05	גרסה חדשה	בעלים	תקף, קובץ JWT קיים	encrypt → POST /versions → FileVersion doc
UC-06	מחיקת קובץ	בעלים	תקף JWT	unlink disk → delete File + Permissions
UC-07	Audit Logs	מנהל	תקף JWT	GET /admin/audit-logs → display
UC-08	חסימת מכשיר	מנהל	תקף JWT	POST /admin/block → BlockRule → authRequired blocks

15.3 מבני נתונים בשימוש

- MongoDB Documents: JSON-like objects — מאפשרים nested objects (metadata, arrays)
- Array (JavaScript): openPorts[], permissions[] — הרשאות, רשימות פורטים פתוחים
- Map (JavaScript): wsMap = Map<userId, WebSocket> — userId מיפוי ל WebSocket לחיבור
- ArrayBuffer: WebCrypto — encrypt/decrypt ייצוג בינארי של קבצים ב
- Queue (Python): numpy arrays של timestamp counts לחישוב Z-score ו-EWMA

15.4 חישוב יעילות האלגוריתם

הערה	סיבוכיות מקום	סיבוכיות זמן	פעולה
גודל הקובץ n	$O(n)$	$O(n)$	AES-256-GCM encrypt
AES = 32 bytes מפתח	$O(1)$	$O(1)$	RSA-OAEP wrap key
streaming hash	$O(1)$	$O(n)$	SHA-256 hash
k = מספר רשומות AuditLog	$O(k)$	$O(k)$	Z-score calc
n samples, t trees	$O(n \cdot t)$	$O(n \cdot t)$	Isolation Forest
index על userId עם	$O(1)$	$O(\log n)$	MongoDB query

15.5 הקשרים בין היחידות השונות

User $\longleftrightarrow$ Session (1:N) — sessions כל משתמש יכול לפתוח מספר
 User $\longleftrightarrow$ File (1:N) — משתמש בעל קבצים
 User $\longleftrightarrow$ AuditLog (1:N) — כל פעולה מתועדת
 File $\longleftrightarrow$ Permission (1:N) — קובץ יכול להיות משותף למספר משתמשים
 File $\longleftrightarrow$ FileVersion (1:N) — קובץ יכול לקבל גרסאות
 User $\longleftrightarrow$ Device (1:N) — מכשירים של המשתמש

15.6 עץ מודולים

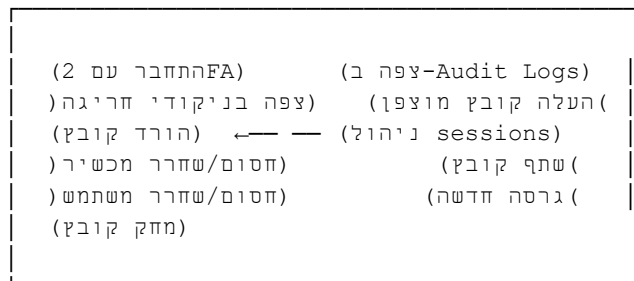
```

SecureSync/
├── Client/src/
│   ├── App.jsx           (main component, state machine: login|verify|files)
│   ├── api.js            (API calls wrapper)
│   ├── crypto/crypto.js  (WebCrypto: encrypt, decrypt, keygen)
│   ├── hooks/useApp.js   (main state management)
│   ├── hooks/useWebSocket.js (WebSocket connection + alerts)
│   └── components/Dashboard.jsx (admin dashboard)
└── Server/Src/
    ├── index.js          (HTTPS server, route mounting, WebSocket)
    ├── models/           (20 Mongoose schemas)
    ├── controller/       (business logic per entity)
    ├── routes/           (Express Router per entity)
    ├── middlewares/auth.js (JWT verify + BlockRule check)
    └── utils/
        ├── audit.js      (logAudit function)
        ├── anomaly.js    (analyzeEvent → Python)
        ├── websocket.js  (sendToUser, broadcast)
        └── pki.js        (RSA sign/verify)
  
```

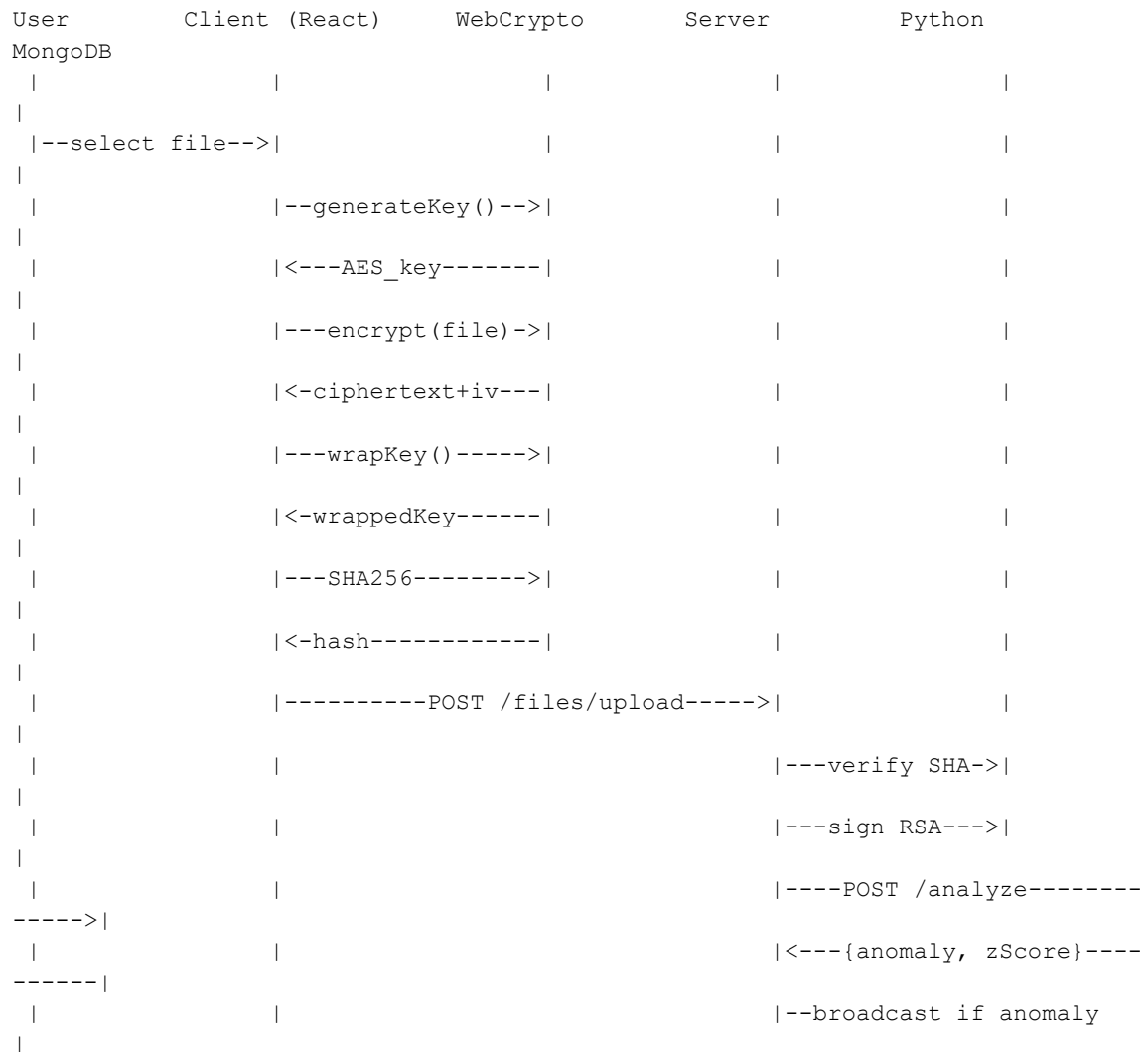
15.7 Use Case Diagram

ה-Use Case Diagram שני Actors: (Admin) משתמש רגיל ומנהל.

[SecureSync System]



↑
[Admin) מנהל]

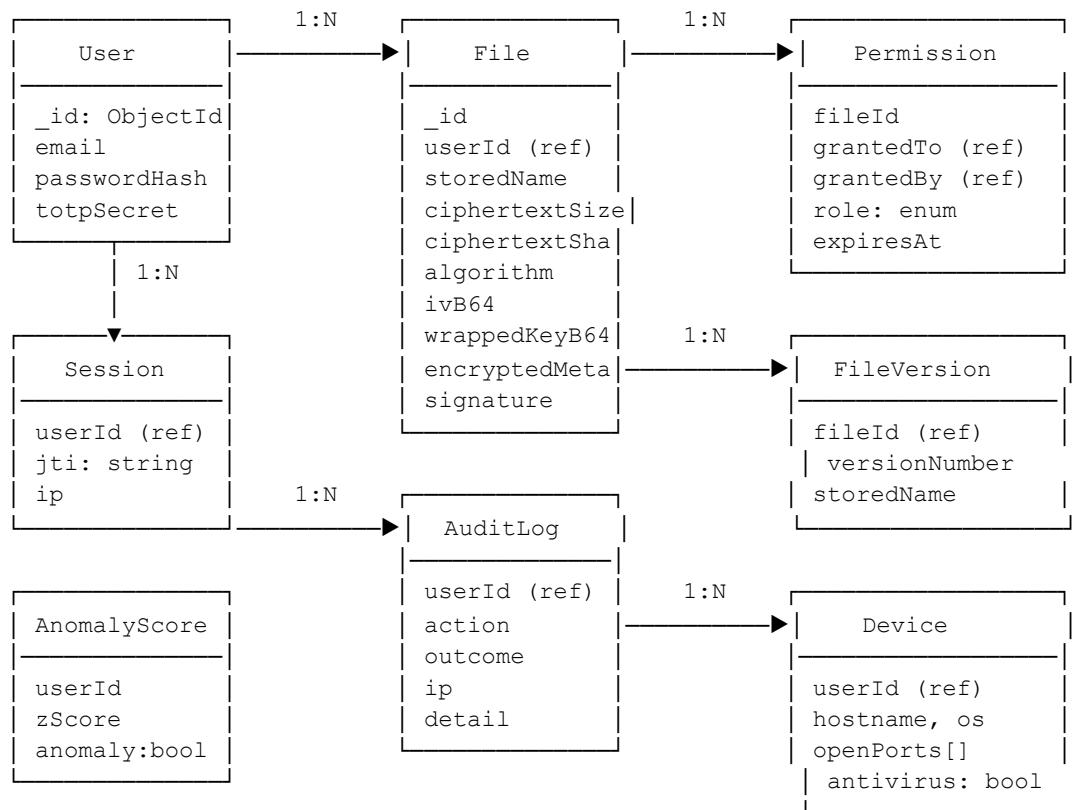


```

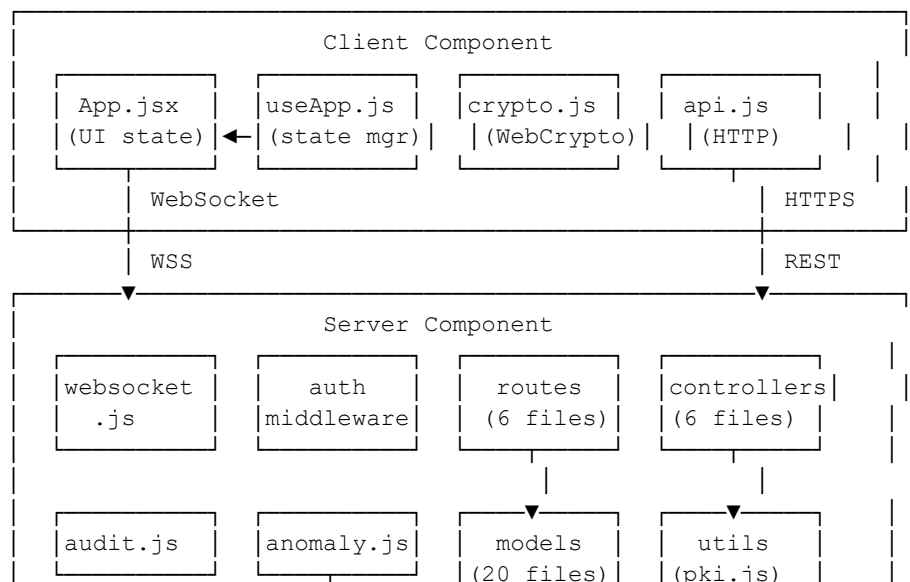
|----->|
|
|<-----{id: fileId}-----|
|

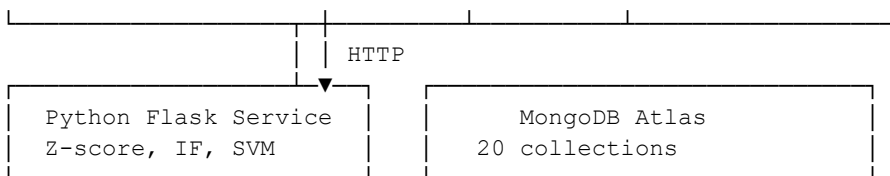
```

15.10 Design Class Diagram



15.11 תרשים רכיבים





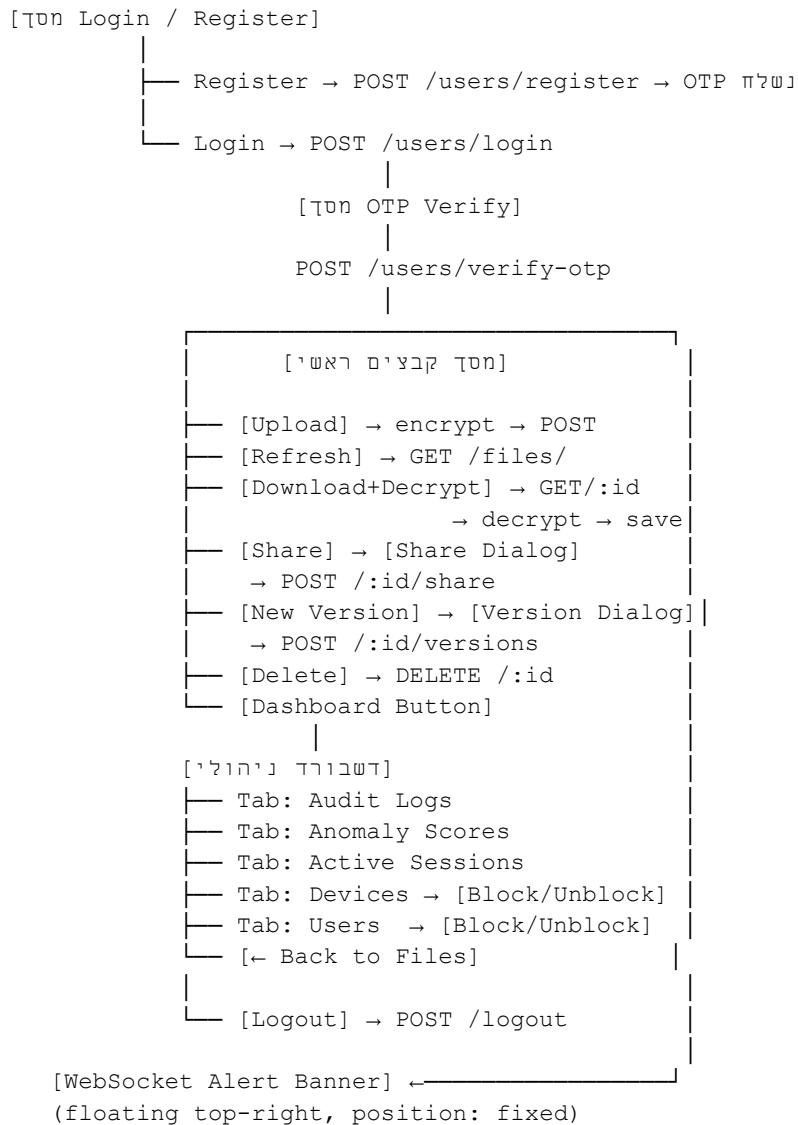
15.12 תיאור הרכיבים (קומפוננטות) המוצעות

מסך כללי לפרטים — כל רכיב עיקרי

פלטים	קלטים	תפקיד	רכיב
תצוגת מסכים	אירועי משתמש, WebSocket events	ראשי ו state ניהול routing	App.jsx
ciphertext, ArrayBuffer	File object, RSA keys	כל פעולות הצפנה/פענוח	crypto.js
state updates	UI אירועי	state ו-API calls ניהול	useApp.js
anomaly alerts	JWT token	חיבור WebSocket + alerts	useWebSocket.js
audit/anomaly/devices tables	JWT, admin role	דשבורד ניהולי 5 טאבים	Dashboard.jsx
req.userId 401/403 או	HTTP request (Authorization header)	JWT + אימות BlockRule	auth.js (middleware)
AuditLog doc in MongoDB	userId, action, outcome, ip, fileId	רישום פעולות	audit.js
AnomalyScore doc + WebSocket broadcast	userId, action	ניתוח חריגות	anomaly.js
signature / boolean	buffer, private/public key	RSA חתימה ואימות	pki.js

משתמש → זרימת מידע: App.jsx → api.js → HTTP → auth middleware → controller → models → MongoDB

16. תרשים מסכים (Screen Flow Diagram)



17. פירוט מסכים

17.1 מסך התחברות /רישום

תפקיד: כניסה למערכת עם אמות אימות ראשוניות

מוצגים שדות שם ואישור "Register" בלחיצה על login ו-register — תיאור: מסך עם שני מצבים הודעות שגיאה מוצגות מתחת לטופס. POST נשלחת בקשת "Login" סיסמה. בלחיצה על

צילום מסך]: יש לצרף צילום מסך של מסך הלוגין בעת הגשה]

17.2 מסך אימות OTP (2FA)

תפקיד: אימות זהות דו-שלבי

מציג את כתובת האימייל שאליה נשלח. (inputMode="numeric") תיאור: מסך עם שדה קוד 4 ספרות "אוטומטי. כפתורי "אמת" ו-"חזרה Auto-focus. הקוד

[בעת הגשה OTP יש לצרף צילום מסך של מסך]: צילום מסך

17.3 מסך ניהול קבצים (ראשי)

תפקיד: ניהול כל הקבצים — העלאה, הורדה, שיתוף, גרסאות, מחיקה

file input אזור העלאה עם (2), Logout ו-Dashboard תיאור: מורכב מ-5 אזורים): 1 (פס עליון עם כפתורי טבלת קבצים עם פעולות, (4), WebSocket-אדום לאנומליות מ banner (3), "Encrypt + Upload" וכפתור לשיתוף וגרסה חדשה dialogs (5)

צילום מסך]: יש לצרף צילום מסך של מסך הקבצים הראשי בעת הגשה]

17.4 דשבורד ניהולי

תפקיד: ניהול מערכת — אודיט, אנומליות, סשנים, מכשירים, משתמשים

תיאור: 5 טאבים

- Audit Logs — User | Action | Result | IP | Time
- Anomaly Scores — User | Action | Z-Score | Details | Time
- Active Sessions — User | IP | Created | Expiry
- Devices — Hostname | OS | IP | Ports | Antivirus | Status | Block
- Users — Email | Name | Registered | Block/Unblock

צילום מסך]: יש לצרף צילום מסך של הדשבורד בעת הגשה]

18. אלמנטי תצוגה

אלמנט	מסך	תפקיד
"Encrypt + Upload" כפתור	קבצים ראשי	מפעיל את תהליך ההצפנה וההעלאה
כפתור "Download+Decrypt"	קבצים ראשי	ומפענח אוטומטית ciphertext מוריד
"Share" כפתור	קבצים ראשי	לשיתוף קובץ dialog פותח
"New Version" כפתור	קבצים ראשי	להעלאת גרסה חדשה dialog פותח
"Delete" כפתור	קבצים ראשי	מוחק קובץ לאחר אישור
אנומליה (אדום) Banner	קבצים ראשי	WebSocket-מציג התראת אנומליה בזמן אמת מ
"Dashboard" כפתור	קבצים ראשי	מנווט לדשבורד הניהולי
"Logout" כפתור	כל המסכים	localStorage מ-JWT ומנקה logout מבצע
שדה OTP (numeric input)	OTP Verify	אוטומטי focus קלט קוד 4 ספרות עם
כפתורי טאב בדשבורד	דשבורד	views: מחליפים בין 5 Audit/Anomaly/Sessions/Devices/Users
Block/Unblock כפתור	דשבורד	חוסם או משחרר מכשיר/משתמש (יוצר/מוחק BlockRule)
role dropdown	Share Dialog	בחירת הרשאה: read / write / admin

19. הודעות למשתמש

סוג	הודעה / Alert	מצב
הצלחה (ירוק)	"OTP sent to your email"	נשלח בהצלחה OTP
שגיאה (אדום)	"Invalid OTP"	שגיאת OTP
שגיאה	"Session expired. Please login again."	פג תוקף JWT
שגיאה 403	"User is blocked"	משתמש חסום
שגיאה 401	"Invalid or expired session"	logout לאחר JWT
שגיאה 400	"File integrity check failed"	אינו תואם SHA-256
שגיאה 403	"File tampering detected"	שגיאת RSA signature
Progress	"Encrypting..."	הצפנה בתהליך
Progress	"Uploading..."	העלאה בתהליך
הצלחה	"Uploaded: [filename]"	קובץ עלה בהצלחה
Alert (WebSocket)	Banner אדום: userId, action, Z-score	(WebSocket) אנומליה זוהתה
שגיאה 403	"Access denied"	ניסיון גישה ללא הרשאה

20. ממשק משתמש

(חיצוני UI framework ללא) ישיר CSS עם React 18-ממשק המשתמש בנוי ב

- שפה: ממשק באנגלית (טבלאות, כפתורים) (עם תמיכה בעברית בשמות קבצים)
- Layout: SPA (Single Page Application) — state-מסך אחד שמשנה את ה
- a90d9aכרטיסים בהירים, כפתורים בצבע #4, (#1a1a2e) עיצוב: רקע כהה
- Responsive: עמודות מסתגלות לגודל המסך
- ברור label ו- placeholder נגישות: כל שדות הקלט עם
- WebSocket Alert Banner: position: fixed, top-right, z-index: 9999, צבע אדום, #c62828
- State Machine: App 3 states מנהל — login | verify | files

21. קוד התוכנית עם תיעוד

21.1 קלטים עיקריים

```
// קלטים מהמשתמש
POST /users/register: { email: string, name: string, password: string }
POST /users/login:    { email: string, password: string }
POST /users/verify-otp:{ email: string, otp: string }

// קלט קובץ (multipart/form-data)
POST /files/upload:
  file:          binary (ciphertext)
  ivB64:         string (base64, 12 bytes IV)
  wrappedKeyB64: string (base64, RSA-wrapped AES key)
  encryptedMetaB64: string (base64, encrypted metadata)
  metaIvB64:     string (base64, metadata IV)
  ciphertextSha256B64:string (base64, SHA-256 hash)
  algorithm:     "AES-GCM"

// קלט שיתוף
POST /files/:id/share: { email: string, role: "read"|"write"|"admin" }

// קלט Python /analyze
POST /analyze: { userId: string, action: string }
```

21.2 פלטים עיקריים

```
// תגובות שרת
POST /users/verify-otp → { token: "eyJ..." } // JWT
GET /files/            → [ { _id, storedName, encryptedMetaB64,
                           wrappedKeyB64, ivB64, algorithm,
                           metaIvB64, createdAt, isOwner } ]

GET /files/:id/download →
  body:    binary ciphertext
  header: x-securesync-meta: base64(JSON({wrappedKeyB64, encryptedMetaB64,
metaIvB64}))

// תגובה Python /analyze
{ "anomaly": bool, "zScore": float, "ewma": float,
  "details": string, "score": float }
```

21.3 פונקציות חשובות /קריטיות

```
// crypto.js – encryptFileWithWrappedKey(file)
// AES עם RSA-OAEP עטיפת מפתח, metadata-הצפנת קובץ ו
async function encryptFileWithWrappedKey(file) {
  const aesKey = await crypto.subtle.generateKey({name:"AES-
GCM",length:256},true,["encrypt","decrypt"]);
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const ciphertext = await crypto.subtle.encrypt({name:"AES-
GCM",iv},aesKey,await file.arrayBuffer());
  const metaJson = JSON.stringify({name:file.name, type:file.type,
size:file.size});
  const metaIv = crypto.getRandomValues(new Uint8Array(12));
  const encMeta = await crypto.subtle.encrypt({name:"AES-
```

```

GCM",iv:metaIv},aesKey,
    new TextEncoder().encode(metaJson));
    const rsaKeyPair = await getOrCreateRsaKeyPair();
    const wrappedKey = await
crypto.subtle.wrapKey("raw",aesKey,rsaKeyPair.publicKey,"RSA-OAEP");
    const hash = await crypto.subtle.digest("SHA-256",ciphertext);
    return { ciphertext, iv, wrappedKey, encMeta, metaIv, hash };
}

// auth.js - authRequired middleware
// בודיקת JWT + בודיקת jti + בודיקת נחונים BlockRule
async function authRequired(req, res, next) {
    const token = req.headers.authorization?.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const session = await Session.findOne({ jti: decoded.jti });
    if (!session) return res.status(401).json({error: "Invalid or expired session"});
    const blocked = await BlockRule.findOne({userId: decoded.userId, active: true});
    if (blocked) return res.status(403).json({error: "User is blocked"});
    req.userId = decoded.userId;
    next();
}

// anomaly.js - analyzeEvent(userId, action)
// Python service תוצאה + broadcast מא anomaly
async function analyzeEvent(userId, action) {
    const res = await fetch('http://localhost:5001/analyze', {
        method:'POST', headers:{'Content-Type':'application/json'},
        body: JSON.stringify({userId, action})
    });
    const result = await res.json();
    await AnomalyScore.create({userId, action, zScore:result.zScore, anomaly:result.anomaly,...});
    if (result.anomaly) broadcast({type:'anomaly', userId, action, ...result});
}

```

22. תיאור מסד הנתונים

מסד הנתונים: MongoDB Atlas — NoSQL document store. 20 Mongoose schemas.

ERD — יחסים בין ישויות:

User —< Session (1:N) — sessions משתמש יכול לפתוח מספר
 User —< File (1:N) — משתמש בעלים של קבצים
 User —< AuditLog (1:N) — כל פעולה מתועדת
 User —< Device (1:N) — מכשירים של המשתמש
 User —< KeyPair (1:1) — לכל משתמש RSA זוג מפתחות
 File —< Permission (1:N) — קובץ משותף למספר משתמשים
 File —< FileVersion (1:N) — גרסאות של קובץ

(Collections) פירוט טבלאות:

חובה	סוג נתונים	מפתח ראשי / זר	עמודות עיקריות	אוסף (Collection)
email ✓	String, String, String	_id (ObjectId)	_id, email, name, passwordHash, totpSecret	users
userId, storedName ✓	ref, String, Number, String	userId → users	_id, userId, storedName, ciphertextSize, algorithm, ivB64, wrappedKeyB64, encryptedMetaB64, ciphertextSha256B64, signature	files
fileId, grantedTo, role ✓	ref, enum(read/write/admin)	fileId→files, grantedTo→users	_id, fileId, grantedTo, grantedBy, role, expiresAt, revokedAt	permissions
userId, action, outcome ✓	ref, String, String, String	userId → users	_id, userId, action, outcome, ip, fileId, detail	auditlogs
userId, jti ✓	ref, String (unique), Date	userId → users	_id, userId, jti, ip, expiresAt	sessions
userId, action ✓	ref, String, Float, Boolean	userId (ref)	_id, userId, action, zScore, ewma, anomaly, details	anomalyscores
userId, hostname ✓	ref, String[], Boolean, enum(online/offline)	userId → users	_id, userId, hostname, os, ipAddress, openPorts, antivirus, status, lastSeen	devices
active ✓	ref, String, Boolean	userId→users, blockedBy→users	_id, userId, hostname, ipAddress, reason, blockedBy, active	blockrules
fileId, versionNumber ✓	ref, Number, String	fileId → files	_id, fileId, versionNumber, storedName, ciphertextSize, algorithm, ivB64, wrappedKeyB64	fileversions
userId, publicKey ✓	ref, String (PEM)	userId → users (unique)	_id, userId, publicKey,	keypairs

			privateKeyEnc	
--	--	--	---------------	--

לא נעשה. (אוטומטיים createdAt, updatedAt timestamps: true משתמש בתצורת collection כל index: userId, jti שדות עם. (לא תומך בכך MongoDB views או stored procedures שימוש ב (unique), email (unique).

23. מדריך למשתמש

הפעלת המערכת:

```
# 1. הפעלת שרת Node.js
cd C:/fullstack/securesync/Server
npm run dev    (→ HTTPS on port 4000)

# 2. הפעלת Python ML service
cd Server/anomaly_service
python main.py    (→ Flask on port 5001)

# 3. הפעלת קליינט React
cd C:/fullstack/securesync/Client
npm run dev    (→ Vite on port 5173)

# 4. גישה דרך הדפדפן
https://localhost:5173
(בשורת הכתובת "thisisunsafe" ← "Advanced" בפעם הראשונה: לחץ)
```

שימוש במערכת:

1. "Register" שם, סיסמה → לחץ, email, רישום: הזן
2. "Verify" בדוק אימייל → הזן קוד 4 ספרות → לחץ: OTP
3. "Encrypt + Upload" בחר קובץ → לחץ → "Choose File" העלאת קובץ: לחץ
4. ליד הקובץ → הקובץ המפוענח מורד "Download+Decrypt" הורדת קובץ: לחץ
5. "Share File" לחץ → email + role הזן → "Share" שיתוף: לחץ
6. "Upload" בחר קובץ חדש → לחץ → "New Version" גרסה חדשה: לחץ
7. ניווט בין הטאבים → "Dashboard" דשבורד: לחץ
8. נמחק מהשרת session-ה — "Logout" יציאה: לחץ

בדיקות והערכה 24.

בדיקות פונקציונליות 24.1

#	תרחיש	פעולה	תוצאה צפויה	תוצאה בפועל
1	רישום משתמש	POST /users/register	נשלח לאימייל OTP	✓
2	שגוי OTP	POST /users/verify-otp	שגיאה 400	✓
3	נכון OTP	POST /users/verify-otp	מתקבל JWT	✓
4	העלאת קובץ	POST /files/upload	{id: fileId}	✓
5	רשימת קבצים	GET /files/	מערך קבצים	✓
6	הורדת קובץ	GET /files/: id/download	ciphertext + header	✓
7	פענוח קובץ	WebCrypto client-side	קובץ מקורי	✓
8	שיתוף קובץ	POST /files/: id/share	נוצר Permission	✓
9	גישה לקובץ משותף	GET /files/ (user B)	קובץ מופיע	✓
10	גרסה חדשה	POST /files/: id/versions	גרסה נשמרת	✓
11	מחיקת קובץ	DELETE /files/: id	ומדיסק DB-נמחק מ	✓
12	Logout	POST /users/logout	נמחק Session	✓

בדיקות אבטחה 24.2

#	תרחיש	תוצאה צפויה	תוצאה בפועל
1	גישה לקובץ של משתמש אחר	403 Access denied	✓
2	פג תוקף JWT	401 Unauthorized	✓
3	JWT אחרי logout	401 Invalid session	✓
4	לא תואם SHA-256	400 Hash mismatch	✓
5	שגיאה RSA signature	403 Tampering detected	✓
6	משתמש חסום	403 User is blocked	✓
7	מכשיר חסום	403 Device is blocked	✓
8	JWT בקשה ללא	401 No token	✓

Anomaly Detection בדיקת 24.3

שלבי הבדיקה:

1. baseline-היסטוריים כ AuditLog הכנסת 59 לוגי
2. רבות בזמן קצר download ביצוע פעולות
3. Python service חישוב $Z\text{-score} = 3.44$ (מפל סף 2.0)
4. $\text{anomaly} = \text{true} \rightarrow \text{WebSocket broadcast}$
5. ✓ אדום הופיע בקליינט בזמן אמת Banner

✓ ומוצג בדשבורד DB-נשמר ב AnomalyScore 6.

25. מסקנות

כל 9. (Defense in Depth) מממש מערכת אבטחת קבצים ארגונית מלאה עם 7 שכבות הגנה SecureSync. מטרת הפרויקט הושגה.

לקחים עיקריים:

- עוצמתי — הצפנה חזקה בדפדפן ללא ספריות חיצוניות WebCrypto API
- חייב להיות מוצפן בנפרד metadata דורש תכנון קפדני — כל Zero-Knowledge
- False Positives ל-False Negatives tuning דורש Anomaly Detection — בין
- Defense in Depth — לא שכבה אחת מושלמת
- concurrency מתאים מאוד לשרת קבצים — תפוקה גבוהה עם Node.js async I/O
- schema migrations בלי models גמיש לפיתוח מהיר — MongoDB 20

26. פיתוחים עתידיים

עדיפות	תיאור	פיתוח
גבוהה	Certificate Authority + mTLS Agent בין פנימי	PKI מלא
גבוהה	React Native app עם WebCrypto polyfill	Mobile App
גבוהה	AWS/Azure deployment עם TLS (Let's Encrypt)	Deployment לענן
בינונית	זרימת שחזור סיסמה מאובטחת	Password reset
בינונית	למשמש quota-מגבלת גודל ו	File size quota
נמוכה	לניטור אנטיוירוס, פורטים OS-ב agent תוכנת	Endpoint Agent
נמוכה	רשת switch/router-חסימת מכשיר ב	Real-time router blocking
נמוכה	לאחר אימון Isolation Forest/SVM שמירת מודלי	ML model persistence

27. ביבליוגרפיה

1. MDN Web Docs — Web Crypto API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API
2. NIST SP 800-38D — Recommendation for Block Cipher Modes of Operation: GCM. 2007.
3. RFC 7519 — JSON Web Tokens (JWT). IETF. 2015.
4. OWASP Top 10. <https://owasp.org/www-project-top-ten/>
5. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys.
6. Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. IEEE ICDM.
7. Schölkopf, B. et al. (2001). Estimating the Support of a High-Dimensional Distribution. Neural Computation.
8. MongoDB Mongoose Documentation. <https://mongoosejs.com/docs/>
9. scikit-learn Documentation. https://scikit-learn.org/stable/modules/outlier_detection.html
10. Express.js Documentation. <https://expressjs.com/>
11. React Documentation. <https://react.dev/>
12. Vite Build Tool Documentation. <https://vitejs.dev/>
13. ws — WebSocket Node.js. <https://github.com/websockets/ws>
14. jsonwebtoken npm. <https://github.com/auth0/node-jwt-token>
15. bcrypt npm. <https://github.com/kelektiv/node.bcrypt.js>
16. Nodemailer Documentation. <https://nodemailer.com/>